第三章 内存管理
一 内存分配与回收-传统存储管理
1.1连续分配
1.2非连续分配
1.2.1 基本分页存储管理-（含快表
二 基本分段存储管理
三 内存空间的扩充
四 请求分页管理方式
1. 最佳置换算法（OPT）
2. 先进先出置换算法（FIFO）-进程分配物理块增加，会产生缺页增加（belady异常）
3.LRU（ least recently used） 最近最久未使用
判断缺页率：在内存中不存在，则缺页
四 习题
1. 内存保护目的-不越界，需要OS+硬件（重定位+上下限register）支持
2 **硬件地址变换的机构** 与 动态重定位装入
3. 固定分区分配-预先固定
4 动态分区分配-作业（静）与进程（动）的区别
☘5 基于顺序搜索的动态分区分配算法-首次（地址↑），最佳（容量↑），最差（↓）
☘6 基于索引搜索的动态分区分配算法
7 页式 与段式
8. 程序的链接和装入过程
🥕何时形成完整的逻辑地址？-装入
✅链接：
0编址空间与xx有关
1内存的物理存储速度由xx决定
2 存储管理的目的是-1）为进程分配内存空间，2）回收被进程释放的内存空间，3）提高内存的利用率
3 主存储器的访问（字/字节）和分配单位（块/页，段）
4 段式-方便【共享】+按逻辑存储
5.是否支持多道程序设计-连续分配（no） 固定/可变分区 分页式（yes）
6.段式 段页式访问内存次数 2次 3次
7 段页式存储时-内存地址的结构是二维的?
8分页存储管理

第三章 内存管理

一 内存分配与回收-传统存储管理

1.1连续分配

单一连续分配
固定分区分配
动态分区分配

1.2非连续分配

1.2.1 基本分页存储管理-（含快表

今天复习的是具有快表的地址变换结构，主要是指快表和直接访问内存
为了使访问内存的速度更快一些，所以引入了快表，如果直接访问快表的话时间是1us，但如果访问内存当中的列表项再通过列表项访问内存，那么因为访问内存的时间是100 毫秒，所以耗时更长所以我们访问内存就出现哪种情况
第一种情况就是 先访问快表，然后快表中找到了访问内存的地址，然后再通过这个地址访问内存
所耗费的时间是：1us 访问快表，再加上100us访问内存
第二种情况就是直接访问内存当中的慢表，通过慢表找到访问内存地址，然后再通过这个地址访问内存
所耗费的时间是100us 访问内存 通过列表项找到内存地址，再加上100us 访问内存
由此可见查询快表的速度比查询列表的速度快很多，因此我们只要加上快表命中就可以节省很多时间，一般来说，因为局部性原理， 快表的的命中率可以达到90%以上

那么完整的地址更换过程就是
CPU给出逻辑地址，
页表是一个映射表，包含页号和页内偏移量，主要是逻辑地址到物理地址的映射
页表包含快表，一级页表，二级页表，三级页表等
多级页表机制，各级页表大小不能超过一个页面
比如
注：快表（tlb ）如果快表中有查找的内存地址，直接访问内存地址一次，拿到所需地址即可
1️⃣快表 访问内存一次（如果快表中有，如果没有就是访问内存两次）
2️⃣一级页表 访问内存两次
3️⃣二级页表 访问内存三次
4️⃣三级页表 访问内存四次 前三次访问页表 拿到内存地址 第四次访问内存地址拿到所需地址
主存是物理地址

二 基本分段存储管理

分段：二维的 需要给出段表长 段号？
分页：一维的 只需要页内偏移量

分段对用户可见的，比如C盘和地盘，但分页对用户是不可见
分段：内存保护 就是这个段，他允许访问，然后另一个段就可以不允许访问，
分页：因为是一个整体，这个页能访问那个页不能访问，不好实现。

分段的话，是逻辑上的；
分页的话，是物理上的；

需要特别注意的，是分段的话，他是必须给出段长， 页表的页长都是固定的，所以不需要显示的记录，但段表的段长他是不固定的，所以需要**在段表当中显式的记录段长**，比如0号段长是7K ，1号段长是3K
而且**必须进行越界判断**，就是要判断这个段 需要判断段号是否越界
各个段表相在内存中占的空间是相同的，也就是各个段表项的的长度是相同的
分页存储有页表，分段存储也有类似的段表，
段表当中记录段号、段长以及基址（起始地址），
每个段之间是连续存储的，但段和段之间不一定是连续存储的，如果要找到这个段，那么就必须知道他的段号，
有点像电脑里的C盘和地盘，你必须知道他在哪个盘，然后才能知道在这个盘里找到具体的位置

列表记录的是逻辑位置和各个页框之间的映射关系
段表记录的是各个逻辑段到实际的物理地址的映射关系
kmg

段页式存储-一共访问三次内存
就是将地址空间按照自身逻辑关系划分成若干个段，在每个段中划分为相等的页面，
将内存空间分为与页面大小相等的一个个内存块一块儿， 系统以块为单位进行内存分配

逻辑地址由段号，页号，页内偏移量三部分组成，

分页管理没变，页表记录的都是页号和物理地址的起始关系，
但是段表变了， 段表长度是相同的，所以也可以是隐藏的
访问内存次数：
第一次是访问段表，第二次是访问页表，第三次是访问内存
根据段表中记录的列表长度，检查页号是否越界
如果快表命中，段页式仅需要一次访存（不需要访问段表和页表，即直接访问快表加内存，

每个段对应一个段表项，各段表项长度相同，由段号(hidden)，页表长度，页表存放地址组成
每个页对应一个页表项，各列表项长度相同，由页号(hidden)，页面存放的内存块号组成

三 内存空间的扩充

1覆盖技术
2交换技术
3虚拟存储技术

1 一次性：传统的存储管理方式会造成作业很大时，不能全部一次性装入内存找导致大作业无法执行；当大作业要求运行时内存无法容纳所有作业，因此只能容纳少量作业，导致多道程序并发性 下降
2 驻留性：内存中驻留留了大量的暂时用不到的数据，造成了内存资源的浪费

之前讲了
根据时间 局部性和空间局部性的原理
时间局部性是这条指令直接完之后不久之后可能再次执行再次访问
空间局不行是一旦访问了某个组织党员在不久之后，他附近的存储单元很有可能被访问（因为很多数据在内存当中都是连续存放的，并且程序的指令也是顺序的在内存当中存放的）

根据局部性原理，在程序装入时，可以将很快会用到的部分装入内存，暂时用不到的放在外存。
在程序执行过程当中，
当所访问的信息不在内存中，由操作系统负责，将所需信息从外存调入内存，然后继续执行程序。
当内存空间不够，由操作系统负责，将内存当中暂时用不到的信息交换出外存。
在操作系统中管理一下，在用户看来似乎是一个比实际内存大得多的内存，这就是虚拟内存那么如何实现虚拟内存技术呢?
如何使用连续分配的话会实现不方便，在虚拟内存的实现需要建立在离散分配的内存管理的基础之上

为了满足上面的这两个功能，操作系统需要增加两个新功能
第一个是请求调页（请求调段）功能 -调入内存
第二个是页面（段）置换功能功能 -换出内存

四 请求分页管理方式

页表机制：
在基本分页的基础上，还增加了几个表项
状态位： 表示页面是否在内存中
访问字段：记录最近被访问过几次或记录上次访问的时间，供置换算法选择换出页面时参考
修改位： 表示页面调入内存后是否被修改过， 只有修改过的页面才需要在置换时写回外存
外存地址： 页面在外存中存放的位置

缺页中断机制：
找到页表项后检查页面是否在内存中，若没在内存，则产生缺页中断
缺页中断处理中，需要将目标页面调入内存， 有必要是还要换出页面
缺页中断 属于🚩**内中断**， 属于内中断当中的“故障”， 即可能被系统修复的异常
一条指令在执行过程中可能产生多次缺页中断

1. 最佳置换算法（OPT）

替换从当前到后面（右边） 最久访问的页面
比如在第四个访问2时，内存块中是701，此时替换7 ，因为7在倒数第三个才访问，把7换成2

2. 先进先出置换算法（FIFO）-进程分配物理块增加，会产生缺页增加（belady异常）

先进先出（FIFO）页面置换算法的核心是淘汰 “最早进入内存” 的页面。结合图中例子（系统给进程分配 3 个内存块，页面引用串是 3,2,1,0,3,2,4,3,2,1,0,4），看到 “访问 1” 这一步时：

这里要标记进入内存的顺序 以免出现图片中的错误疑问！！！

关键逻辑：

当前内存中的页面是「4（内存块 1）、3（内存块 2）、2（内存块 3）」。按照 FIFO 规则，要找最早进入内存的那个页面 —— 其中 “3” 是更早进入内存的（比 “2” 和 “4” 进入得早），所以会淘汰 “3”，把新页面 “1” 放入 “3” 原来所在的「内存块 2」。

3.LRU（ least recently used） 最近最久未使用

从当前项 往前（左）逆向检查 ，替换最后一个（最左）页面

时钟clock置换算法

页号 内存块号 状态位 外存地址
访问位: 初始为0 访问后变为1

修改位：初始为0 置换后变为1

判断缺页率：在内存中不存在，则缺页

简单的clock 最多两轮扫描
改进的clock，最多四轮扫描
访问位 修改位
从当前位置开始扫描到
第一轮替换0，0，不修改标志位；；没访问过，且没修改过

第二轮替换0，1，扫描过的帧 访问位设为0；；没访问过，但修改过
第三轮替换0，0，不修改标志位；；最近访问过，但没修改过

第四轮替换0，1；；最近访问过，修改过

四 习题

1. 内存保护目的-不越界，需要OS+硬件（重定位+上下限register）支持

1. ✅**内存保护**的目的：保证访问不越界，由🚩操作系统+硬件系统支持，

硬件系统的支持指的是

🚩重定位寄存器（基地址寄存器），界地址寄存器=》用于越界检查🧐

🚩上下限寄存器=〉用于越界检查🧐

单CPU时，每个时刻只有一个进程在运行，所以整个系统只需要一个重定位寄存器，并不需要每个进程，或者每个程序都有一个，当这个进程上处理机的时候，我们就更新这个重定位寄存器；

被访问单元的物理地址=逻辑地址+重定位寄存器的值（一般来说）

2. ✅需要**硬件地址变换的机构**：即硬件地址变换机构就是问逻辑地址转化为物理地址需要硬件支持

产生物理地址是在装入阶段；

2 硬件地址变换的机构 与 动态重定位装入

1）绝对装入是：编译之后直接产生物理地址

2）静态重定位装入：在装入时一次性完整产生完整的物理地址（进程数据一旦确定位置后不可移动）-》单一连续分配和固定分区分配；

3）动态重定位装入：程序/指令真正**执行时**才转化为物理地址=〉页式存储管理，动态分区分配页式虚拟存储管理三种，需要🚩硬件地址变换机构

绝对装入1和静态重定位2装入，是不存在物理地址变换的过程，所以它不需要硬件地址变换机构；

为保证一个系统在储存中被改变存放位置后，仍能正确执行，应该采用动态重定位，因为动态重定位是程序真正执行时才将逻辑地址转化为物理地址，所以改变存放位置的时候，根本不影响动态重定位。

Exe装入内存

操作系统实现**分区存储管理代价最小**，既不需要段表，也不需要页表。

3. 固定分区分配-预先固定

分为分区大小相等和分区大小不等，🚩但预先是固定的

动态分区分配存储管理中-》<u>是在运行过程中，**在作业🚩装入时**动态建立的</u>

4 动态分区分配-作业（静）与进程（动）的区别

作业（Job）：静态的用应用软件一个程序都可以称为一个作业

进程（Process）：动态的，一个正在运行的程序

🍀5 基于顺序搜索的动态分区分配算法-首次（地址↑），最佳（容量↑），最差（↓）

首次适应算法：不需要对空闲列表进行排序的分配算法，按空闲分区**地址递增**的次序链接

最佳适应算法：空闲分区按容量/**长度递增**的次序进行排列 ↑，找和他数据最近的比他大一点的空间分区

最差适应算法(最坏适应算法):空闲分区按容量/**长度递减** ↓的次序进行排列

邻近适应算法： 按空闲分区地址递增的次序链接连成一个循环链表

🍀6 基于索引搜索的动态分区分配算法

快速适应算法

伙伴算法

哈希算法 哈希冲突

7 页式 与段式

7.1 **页式管理**-提高内存利用率

页表是OS操作系统建立

页式存储管理所划分的页面大小必须相同，而确定页面大小是根据CPU的架构（也就是硬件层面）和操作系统决定，与外存和内存的大小没有关系。

并不是依据CPU的地址结构决定，CPU地址线是32位的话，根据划分完页框号（页面号）占几位，剩下的是页内偏移量，也就是页面大小；我们是确定页面大小之后，才划分出了CPU的地址结构。

7.2 **段式管理**-共享段表 实现多个进程共享同一段代码/数据

一个进程n个段，有n个段表项，只有一个段表

分页存储管理除了分配给用户进程以外，还要留出一部分区域存放页表，这部分区域是不能给进程直接使用的，所以如果采用分页管理，可以给用户提供的物理地址的大小是要刨除掉页表

同样的分段存储管理提供给用户的**物理地址**大小也是要抛除段表，剩下的才是有可能**给用户分配的物理地址**，

若页表和段表大小是未知的，那么就不知道分配的物理地址空间的大小

可重入程序（代码）：是指能够**被多个进程共享**的**代码**段，它不属于临界资源，其中的数据不能被修改

采用可重入程序是通过**减少对换数**量的方法来改善系统性能的

7.3 装入

采用页式存储的时候，一个程序如何分页是在装入作业（exe装入内存）时由**操作系统**决定的。

CPU将虚拟地址划分为页号和页内偏移量，然后通过硬件中的**页表寄存器**和内存管理单元（MMU），将页号转换为 **物理地址再拼上页内偏移量**，最终得到的内存物理地址，这个过程是由🚩硬件自动完成，不需要操作系统或者其他软件的干预。

采用段式存储的时候，一个程序如何分段是在用户**编程**的时候决定

8. 程序的链接和装入过程

🥕何时形成完整的逻辑地址？-装入

程序员=》预处理-》编译/汇编=〉一组目标模块=》+库函数-》链接（成一个完整模块）=〉装入（形成**完整的逻辑地址**/虚地址/相对地址）-》到内存（物理地址/实地址/绝对地址）

✅需要**硬件地址变换的机构**：即硬件地址变换机构就是问逻辑地址转化为物理地址需要硬件支持

产生物理地址是✅装入阶段；

1）绝对装入是：编译之后直接产生物理地址

2）静态重定位：在装入时一次性完整产生完整的物理地址（进程数据一旦确定位置后不可移动）-》单一连续分配和固定分区分配；

3）动态重定位：程序/指令真正**执行时**才转化为物理地址=〉页式存储管理，动态分区分配页式虚拟存储管理三种，需要🚩硬件地址变换机构

绝对装入1和静态重定位2装入，是不存在物理地址变换的过程，所以它不需要硬件地址变换机构；

为保证一个系统在储存中被改变存放位置后，仍能正确执行，应该采用动态重定位，因为动态重定位是程序真正执行时才将逻辑地址转化为物理地址，所以改变存放位置的时候，根本不影响动态重定位。
Exe装入内存

✅链接：

静态链接：在**装入内存之前**就产生了完整的逻辑地址

装入时动态链接：在**装入内存时**就产生了完整的逻辑地址

运行使动态链接：程序要**执行时**产生相应的逻辑地址

零散知识：

0编址空间与xx有关

在虚拟内存系统中，磁盘空间无限大，作业并不一定拥有任意大的空间，编址空间还与🚩地址线位数有关。

如果地址线为32bit，那么他最大就能寻址2^ 32=4G

1内存的物理存储速度由xx决定

内存的物理存储速度由🚩内存的硬件决定（芯片决定），与存储管理是没有关系的

2 存储管理的目的是-1）为进程分配内存空间，2）回收被进程释放的内存空间，3）提高内存的利用率

页式存储，如果这一个页当中一部分允许被访问，另一部分不允许被访问，那么他就不如段式存储，存储共享和访问更加方便

3 主存储器的访问（字/字节）和分配单位（块/页，段）

主存储器的**访问**以字或者是字节为单位，

主存储器的**分配**以块/页或者是段为单位

分页存储或者分段存储或者段页式存储

CPU-》字-〉主存-》块-〉磁盘

4 段式-方便【共享】+按逻辑存储

段式方便共享（因为按逻辑功能模块存储）

由于分页存储并不是按照逻辑来进行划分，所以在内存的共享和保护的时候不是很方便，**在某一个页面上，可能出现一部分允许共享，另一部分不允许共享**

而分段存储的话是按**逻辑功能模块（如按函数，变量存在不同的段中）**，用户编程更方便，程序的可读性更高。如进程A的话它可以访问某个段名为D的二号段，而进程 A并不能访问某个段名为X的一号段。

举例：段式存储的话如果有三个段，

第一个段的话存的是main函数，

第二个段存的是某个子函数，

第三个段存的是保存的全局变量，

如果执行if else语句走一个函数，走不到else里面，这个段名里面存的某个子函数都不必访问。

5.是否支持多道程序设计-连续分配（no） 固定/可变分区 分页式（yes）

1.） 用户连续分配管理：在我们的内存当中，除了用于存放操作系统数据这片区域之外，剩下的全部区域都是能存放一个用户的一道程序的数据，并且只能连续存放。所以当用户连续存储是不支持多道程序所设计的，

2.）固定分区分配方式：操作系统把内存空间划分为一个固定的分区，那么每个分区都可以给它分配一个进程，所以它是支持多道程序设计的。

3.）可变分区分配操作系统，从目前剩余的分区当中摘出来一部分给某一个进程，如果此时还有剩余的空闲空间，那么可以从这些剩余的当中再摘除一部分给另一个进程，称为可变分区或者是动态分区，它也是支持多道程序设计的。

4.）分页式存储管理，内存被划分了成了多个**页框/物理块**，这些页框可以分配很多个进程，也是支持多道程序设计的

页框号=物理块号

6.段式 段页式访问内存次数 2次 3次

段式存储： CPU每次从内存中 取一次数据需要**两**次访问内存

第一次段表存储在内存中，需要进行访存

段式分配，逻辑地址由段号和段内偏移量组成，段表当中存储的是目标段在内存当中的起始地址，而段表是存储在内存当中的，第一次访存是访问在内存当中的段表，获取目标段在内存当中的起始地址，然后拼接起始地址加段内偏移量形成物理地址，通过这个物理地址再次进行访问，内存拿到目标单元

第二次目标单元存储的内存，当中需要进行访存

段页式存储：CPU每次从内存中 取一次数据需要三次访问内存

段页式分配逻辑地址是由段号页号和页内偏移量组成，首先是
段表存储在内存当中，页表也存储在内存当中，
所以我们通过访问内存当中的段表，拿到目标段对应的页表的起始地址，
然后再次访问内存，通过页表的起始地址拿到目标页面的起始地址，然后拼接页内偏移量形成一个完整的物理地址，
然后第三次访问内存拿到目标内存单元，

所以一共是三次。

7 段页式存储时-内存地址的结构是二维的?

而且不需要存储页号，为什么

段页式存储就是每个进程一张段表，每个段对应一个页表

8分页存储管理

【**每个进程**】拥有一张页表，并不是所有的进程都共享一个页表，但是列表需要常驻在内存当中